

SKILL_12

1. To Execute Large Pipelines, Launch Multiple Jobs, Measure Stability, Monitor Resources, Detect Failures, Analyze 4 Performance

CODE:

```
GNU nano 7.2 pipeline_test.c
#include <stdio.h>
#include <unistd.h>

int main()
{
    for(int i = 1; i <= 5; i++)
    {
        printf("Job %d Running...\n", i);
        sleep(1);
    }

    printf("All Jobs Completed Successfully\n");

    return 0;
}
```

RESULT:

```
top - 05:42:27 up 55 min, 1 user, load average: 0.03, 0.03, 0.00
Tasks: 21 total, 1 running, 20 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.0 sy, 0.0 ni, 100.0 id, 0.0 wa, 0.0 hi, 0.0 si,
MiB Mem : 7784.6 total, 7099.4 free, 502.9 used, 332.5 buff/cache
MiB Swap: 2048.0 total, 2048.0 free, 0.0 used. 7281.7 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+
1	root	20	0	21732	13100	9784	S	0.0	0.2	0:00.82
2	root	20	0	3180	2200	2072	S	0.0	0.0	0:00.02
6	root	20	0	3196	2128	2008	S	0.0	0.0	0:00.00
52	root	19	-1	34056	12556	11480	S	0.0	0.2	0:00.22
101	root	20	0	25288	6700	5152	S	0.0	0.1	0:00.70
124	systemd+	20	0	21352	13188	11064	S	0.0	0.2	0:00.15
127	systemd+	20	0	91044	7988	7024	S	0.0	0.1	0:00.15
181	root	20	0	4248	2676	2424	S	0.0	0.0	0:00.01
182	message+	20	0	9548	5364	4672	S	0.0	0.1	0:00.06
195	root	20	0	17988	8828	7792	S	0.0	0.1	0:00.09
217	syslog	20	0	222524	5976	4612	S	0.0	0.1	0:00.08
238	root	20	0	3128	1964	1832	S	0.0	0.0	0:00.01
254	root	20	0	107052	23112	13660	S	0.0	0.3	0:00.17
407	root	20	0	6680	4580	3808	S	0.0	0.1	0:00.01
450	root	20	0	20336	11476	9352	S	0.0	0.1	0:00.09
451	root	20	0	21188	3520	1804	S	0.0	0.0	0:00.00
475	root	20	0	6084	5320	3644	S	0.0	0.1	0:00.01
792	root	20	0	3184	1112	980	S	0.0	0.0	0:00.00
794	root	20	0	3200	1252	1108	S	0.0	0.0	0:00.06
797	root	20	0	6080	5432	3700	S	0.0	0.1	0:00.05
1086	root	20	0	9308	5688	3504	R	0.0	0.1	0:00.00

REPORT:

In this experiment, multiple jobs were executed simultaneously to study pipeline performance and system stability. Resource usage was monitored using Linux commands, and job execution was verified. The experiment helps in understanding process management, performance analysis, resource monitoring, and failure detection in Linux systems.

2. To Document Architecture, Create Flow Diagrams, Explain Execution Paths, Describe Syscalls, Record Design 4 Decisions, Review Documentation

CODE:

```
GNU nano 7.2
System Architecture

User Input
|
v
Program Execution
|
v
System Calls
(open, read, write, close)
|
v
Output Generation
|
v
Result Display|
```

RESULT:

```
root@SRIVALLI:~# nano architecture.txt
root@SRIVALLI:~# cat architecture.txt
System Architecture

User Input
|
v
Program Execution
|
v
System Calls
(open, read, write, close)
|
v
Output Generation
|
v
Result Display
```

REPORT:

In this experiment, software architecture was documented using a simple flow diagram and execution path description. The system calls used by the program were identified, and design decisions were recorded. This experiment helps in understanding program structure, execution flow, and the importance of proper software documentation.

3. To Organize Source Code, Improve Build Scripts, Add README, Verify Repository Structure, Tag Releases, Validate Deliverables

CODE:

```
GNU nano 7.2 src/main.c
#include <stdio.h>

int main()
{
    printf("Project Running Successfully\n");
    return 0;
}
```

```
GNU nano 7.2 README.md *
Project Title

Description:
This project demonstrates basic project organization and build management.

Files:
- src/main.c
- build.sh
- README.md
```

RESULT:

```
root@SRIVALLI:~# mkdir MyProject
cd MyProject
root@SRIVALLI:~/MyProject# mkdir src
nano src/main.c
root@SRIVALLI:~/MyProject# nano build.sh
root@SRIVALLI:~/MyProject# chmod +x build.sh
root@SRIVALLI:~/MyProject# nano README.md
root@SRIVALLI:~/MyProject# ./build.sh
root@SRIVALLI:~/MyProject# ./project
Project Running Successfully
```

REPORT:

In this experiment, the project source code was organized into folders, a build script was created, and a README file was added for documentation. The repository structure was verified, a Git repository was initialized, and a release tag was created. This experiment helps in understanding project organization, version control, documentation, and software delivery management.

4. TO Create Demo Scenarios, Validate Features, Rehearse Presentation, Demonstrate Pipelines, Demonstrate Signals, Final Project Review

CODE:

```

root@SRIVALLI: ~/MyProject
GNU nano 7.2                                final_demo.c *
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handler(int sig)
{
    printf("\nSignal Received: %d\n", sig);
}

int main()
{
    signal(SIGINT, handler);

    printf("Final Project Demo Running...\n");

    for(int i = 1; i <= 5; i++)
    {
        printf("Pipeline Stage %d Executed\n", i);
        sleep(1);
    }

    printf("Press Ctrl+C to test signal handling\n");

    sleep(10);

    printf("Demo Completed Successfully\n");

    return 0;
}

```

RESULT:

```

root@SRIVALLI:~/MyProject# nano final_demo.c
root@SRIVALLI:~/MyProject# gcc final_demo.c -o final_demo
root@SRIVALLI:~/MyProject# ./final_demo
Final Project Demo Running...
Pipeline Stage 1 Executed
Pipeline Stage 2 Executed
Pipeline Stage 3 Executed
Pipeline Stage 4 Executed
Pipeline Stage 5 Executed
Press Ctrl+C to test signal handling
^C
Signal Received: 2
Demo Completed Successfully

```

REPORT:

In this experiment, demo scenarios were created to validate project features and demonstrate pipelines and signal handling. The program execution was verified, outputs were checked, and the final project review was completed successfully. This experiment helps in understanding project validation, feature demonstration, and final system testing in Linux environments.